Legal AI Chatbot

Complete MLOps Portfolio Project

Fine-tuned GPT-4o-mini • FastAPI • LangSmith • MLflow • Docker • Render

Developer	Krishna
Live API	https://legal-chatbot-a1fb.onrender.com
GitHub	https://github.com/Krishna598-DS/legal-chatbot
Date	May 2026
Model	ft:gpt-4o-mini-2024-07-18:personal:legal-chatbot:Dd52Ybnu

1. Project Overview

This project demonstrates a complete, production-grade MLOps pipeline for a legal domain AI chatbot. Starting from raw data creation through fine-tuning, serving, tracing, containerization, and cloud deployment — every component reflects real-world engineering practices used at top technology companies.

Business Problem

General-purpose LLMs provide inconsistent legal information in varying formats. A fine-tuned domain-specific model provides consistent, professionally formatted legal information with appropriate disclaimers — at lower cost and higher reliability than prompt-engineered general models.

Solution Architecture

- Dataset: 60 legal Q&A; pairs across 6 domains, formatted as OpenAI JSONL
- Fine-tuning: GPT-4o-mini fine-tuned via OpenAI API — loss reduced 84.95%
- Experiment Tracking: MLflow logs parameters, metrics, artifacts per run
- API Server: FastAPI with Pydantic validation, retry logic, streaming support
- Observability: LangSmith traces every LLM call with full input/output/latency
- Frontend: Streamlit chat UI with real-time token and cost tracking
- Containerization: Docker Compose orchestrates API and UI as separate services
- Deployment: Render cloud hosting with health checks and auto-deploy from GitHub

2. Technology Stack

Layer	Technology	Version	Purpose
LLM	OpenAI GPT-4o-mini	Fine-tuned	Legal domain inference
API Framework	FastAPI	0.111.0	REST API server
ASGI Server	Uvicorn	0.29.0	Async request handling
Validation	Pydantic	2.7.1	Request/response validation
Experiment Tracking	MLflow	2.13.0	Training run logging
LLM Tracing	LangSmith	0.1.63	Inference observability
Frontend	Streamlit	1.35.0	Chat user interface
Resilience	Tenacity	8.3.0	Retry with backoff
Tokenization	Tiktoken	0.7.0	Token counting
Logging	Loguru	0.7.2	Structured logging
Containerization	Docker + Compose	29.4.3	Environment isolation
Cloud Deployment	Render	Free tier	Public cloud hosting
Language	Python	3.11.15	Runtime

3. Dataset Preparation & Fine-tuning

Dataset Statistics

Metric	Value
Total Examples	60
Training Examples	54 (90%)
Validation Examples	6 (10%)
Avg Tokens per Example	293
Max Tokens	358
Min Tokens	225
Token Limit (OpenAI)	4096
Format	JSONL (JSON Lines)
Random Seed	42 (reproducible)

Legal Domain Coverage

Domain	Examples	Topics Covered
Contract Law	10	Breach, formation, void/voidable, force majeure, warranties
NDA & Confidentiality	10	Mutual NDA, exclusions, duration, enforcement
Employment Law	10	At-will, wrongful termination, FMLA, WARN Act
Intellectual Property	10	Copyright, patent, trademark, fair use, DMCA
Liability & Indemnity	10	Negligence, product liability, arbitration, class action
General Legal Literacy	10	Civil vs criminal, mediation, power of attorney, due diligence

Training Results

Metric	Value
Base Model	gpt-4o-mini-2024-07-18
Fine-tuned Model ID	ft:gpt-4o-mini-2024-07-18:personal:legal-chatbot:Dd52Ybnu
Fine-tuning Job ID	ftjob-hLTXTU5SlIoZTmmD81YWlNhC
Epochs	3
Total Training Steps	162

Initial Training Loss	1.86
Final Training Loss	0.28
Final Validation Loss	0.87
Loss Reduction	84.95%
Estimated Cost	$0.38
Training Duration	~24 minutes

4. API Documentation

The FastAPI backend exposes a RESTful API with automatic Swagger documentation. All endpoints include Pydantic validation, structured error responses, and request logging.

Method	Endpoint	Description	Auth
GET	/health	Liveness probe — returns server status and model ID	None
GET	/model/info	Returns fine-tuned model details and capabilities	None
POST	/chat	Send a legal question, receive a structured answer	None
POST	/chat/stream	Streaming response — tokens returned as generated	None
GET	/docs	Auto-generated Swagger UI for API exploration	None
GET	/redoc	ReDoc API documentation alternative	None

Sample API Request & Response

```
POST https://legal-chatbot-a1fb.onrender.com/chat
Content-Type: application/json

{"question": "What is an NDA?", "temperature": 0.3, "max_tokens": 500}
```

```
{"answer": "A Non-Disclosure Agreement (NDA) is a legally binding contract...",
"model_used": "ft:gpt-4o-mini-2024-07-18:personal:legal-chatbot:Dd52Ybnu",
"tokens_used": 317,
"response_time_ms": 9151.97,
"timestamp": "2026-05-08T07:40:56.687426"}
```

5. MLOps Pipeline

MLflow Experiment Tracking

- Tracking URI: file:///home/krishna/legal-chatbot/mlflow_runs
- Experiment: legal-chatbot-finetuning
- Run ID: dab63b95fb6c48cb835e4680bf9c9dd7
- Parameters logged: 10 (base model, epochs, dataset stats, inference settings)
- Metrics logged: 7 (training loss at 4 checkpoints, validation loss, cost, steps)
- Artifacts logged: 4 (model config, training JSONL, validation JSONL, run summary)
- Queried programmatically with mlflow.search_runs() to select best model by loss

LangSmith Tracing

- Project: legal-chatbot
- Every /chat API call traced with full prompt, response, latency, token usage
- @traceable decorator wraps inference functions for automatic trace capture
- Enables debugging bad responses by inspecting exact inputs and outputs
- Latency and token trends visible across all production calls
- Integrated with FastAPI service layer — zero changes to business logic

Docker & Deployment

- FastAPI container: python:3.11-slim base, layer-cached pip install, health check
- Streamlit container: separate Dockerfile, depends_on API health check
- Docker Compose: bridge network, service discovery by name (api:8000)
- Render deployment: auto-deploy on GitHub push, environment variables injected
- Health check endpoint pinged every 30 seconds in both Docker and Render

6. Key Interview Questions & Answers

Q1: What is fine-tuning and how does it differ from training from scratch?

Fine-tuning continues training a pre-trained model on domain-specific data. Training from scratch requires billions of tokens and millions in compute. Fine-tuning requires hundreds of examples and costs dollars, leveraging the base model's existing language understanding while shifting its behavior toward the domain.

Q2: What is overfitting and did your model overfit?

Overfitting is when a model memorizes training data instead of generalizing — training loss drops but validation loss rises. My training loss went from 1.86 to 0.28 while validation loss stayed at 0.87. The gap is expected; the model generalizes well as proven by correct responses to unseen legal questions.

Q3: Why FastAPI over Flask?

FastAPI provides async support for concurrent LLM calls, automatic Pydantic request validation (wrong types return 422 automatically), and auto-generated Swagger UI with zero extra code. Flask requires manual validation and documentation. FastAPI is the preferred choice for ML serving APIs at modern tech companies.

Q4: What is LangSmith and why does it matter?

LangSmith is an LLM observability platform that traces every inference call — the exact prompt sent, response received, latency, and token usage. In production, when a user gets a bad answer, LangSmith lets you inspect the exact input/output to determine whether the issue is the prompt, model, or data. Without it you debug blind.

Q5: How would you scale this to 10,000 concurrent users?

Multiple Uvicorn workers behind nginx load balancer. Redis caching for repeated questions. AWS ECS with auto-scaling on CPU and queue depth. CDN for the Streamlit frontend. Async request queuing with Celery for expensive inference calls. Horizontal scaling of the API service with stateless design.

Q6: What is Docker and why did you use it?

Docker packages code, dependencies, and runtime into portable containers that run identically everywhere. Docker Compose orchestrated two services — FastAPI and Streamlit — connected via a bridge network. Streamlit calls FastAPI by service name 'api:8000' not localhost, enabling true container-to-container communication.

Q7: How do you handle OpenAI API rate limits?

Tenacity library with exponential backoff — retry up to 3 times on RateLimitError or APITimeoutError, waiting 4s then 8s then 10s between attempts. Exponential backoff prevents thundering herd where all services retry simultaneously and hit the limit again.

Q8: What would you do differently if rebuilding?

Implement RAG alongside fine-tuning — fine-tuning for style and format, RAG for factual grounding from real legal documents. Add automated evaluation pipeline with semantic similarity scoring before promoting models to production. Use async OpenAI client throughout for better concurrency under load.

7. Project Summary

Component	Status	Details
Dataset Creation	■ Complete	60 legal Q&A pairs, JSONL format, validated
Fine-tuning	■ Complete	Loss 1.86→0.28, 84.95% reduction, $0.38 cost
MLflow Tracking	■ Complete	Run logged, metrics queryable, artifacts saved
FastAPI Server	■ Complete	4 endpoints, Pydantic validation, Swagger docs
LangSmith Tracing	■ Complete	Every call traced, project: legal-chatbot
Streamlit UI	■ Complete	Chat interface, session state, cost tracking
Docker	■ Complete	2 containers, health checks, bridge network
Cloud Deployment	■ Live	https://legal-chatbot-a1fb.onrender.com
GitHub	■ Public	https://github.com/Krishna598-DS/legal-chatbot

■ This project demonstrates production-grade ML engineering:

Data pipeline • Fine-tuning • Experiment tracking • REST API • LLM observability • Containerization • Cloud deployment

Built in 9 days from scratch with zero prior AI knowledge.